Project 1: Processor Scheduling, Performance, and Energy Trade-offs

Graduate Computer Architecture: Project 1 report

Student: Ankit Saini **Student ID:** 242084 **Repository:** _____

Summary. The six scheduling decisions G1–G6 were repaired in the three student files only. The final code passes **40/40 decision cases**, **39/39 short-program comparisons** and the framework tests (**CHECK: PASS**). `make evaluate` produced **12/12 VERIFIED_FOR_THIS_RUN** records. All Study A and B numbers below come from those verified runs at control_scale = 1. **Serial workload (140 ns):** In-order is selected (118.25 ns, 5.695 nJ per period). **Four-stream workload:** Scoreboard is selected at 140 ns (82.00 ns, 5.536 nJ), and Tomasulo is the only feasible design at 70 ns (59.75 ns, 5.798 nJ).

1. Coding tasks (G1–G6)

Only the marked return expressions in `src/inorder_control.cpp`, `src/scoreboard_control.cpp` and `src/tomasulo_control.cpp` were changed (with explanatory comments). No framework, configuration, test, workload or reference file was modified.

Task	Starter expression (wrong / too restrictive)	Repaired expression	Reason
G1	`sources_ready && fu_free && ! older_inflight`	`sources_ready && fu_free`	The controller already offers only the oldest waiting instruction, so launch stays in program order. RAW hazards are covered by the source tags and the same-FU hazard by fu_free. Older work on a *different* FU is not a hazard. Waiting for it serialised the machine.
G2	`row_free && destination_tag == 0 && !any_inflight`	`row_free && destination_tag == 0`	Scoreboard issue stalls only for a structural hazard (FU/row busy) or a WAW hazard (destination already has a pending writer), not because the processor is busy.
G3	`same_register`	`ready_unread && same_register`	WAR applies only while an older reader still needs the *old* value: same register AND value ready but not yet read (Rj/Rk = yes). A reader still waiting on a producer, or one that already captured its operand, does not protect the old value.
G4	`rs_free && fu_free && !any_inflight`	`rs_free`	RS admission and FU launch are separate steps. A free RS is enough to issue, and the FU is checked at launch. Renaming removes WAW, so in-flight work never blocks admission.
G5	`false`	`producer_tag != 0`	A nonzero RAT tag means a newer value is pending, so the source keeps the tag and waits for the broadcast. Tag 0 means the register file value is current and is captured now.
G6	`true`	`current_tag == result_tag`	Only the producer that the RAT still names may write the architectural register and clear the RAT entry. An older, already-renamed result is still broadcast to its consumers but must not overwrite a younger mapping.

Repair order followed the description: G1, then G2–G3, then G5–G6 before G4.

2. Validation

Commands were run from `Project1_ACA`. The before-repair `make demo` output (S1) was saved before any

edit. After the repairs, `make check` and then `make evaluate` were run on the final submitted code.

Check	Result
Decision cases (G1–G6)	40 / 40 passed
Framework tests (parser, ISA, oracle, clock, units, tags, energy, period)	PASS
Short programs: every issue/start/complete/write time, event count, register and energy, 13 programs × 3 models	39 / 39 passed
Overall `make check`	**CHECK: PASS** (finite tests, not a proof for every input)
`make evaluate`	**EVALUATION: PASS**, 12 / 12 records VERIFIED_FOR_THIS_RUN

Environment: Windows 11, Git Bash, g++ 16.2.0 (MinGW-w64 UCRT), GNU Make 4.4.1, Python 3.12.10. On Windows the linker writes `build/sim.exe`. A small wrapper copied the executables to their extensionless names (`build/sim`, which the workflow hashes) and mapped `python3` to the installed Python. No project file was changed for this. The starter workflow itself prints "FILE CONTENT HASH CHECKS: DISABLED"; functional, timing and energy checks remain active.

3. Experiments and architecture selection

Selection rule (Section 6 of the description): first keep only the correct candidates that meet the deadline, then choose the one with the lowest **period** energy. All numbers below are VERIFIED_FOR_THIS_RUN at control_scale = 1, 4 GHz (0.25 ns per cycle), 1,000 ns period, 0.10 mW sleep power.

Study A: Serial workload (iot_serial.asm)

(a) Measured results.

Architecture	Cycles	Time (ns)	Bkg. power (mW)	E_dynamic (nJ)	E_background (nJ)	E_task (nJ)	P_task,avg (mW)	E_period (nJ)	Meets 140 ns?
In-order	473	118.25	20	3.24223	2.365	5.60723	47.42	**5.69541**	Yes
Scoreboard	442	110.50	24	3.59060	2.652	6.24260	56.49	**6.33155**	Yes
Tomasulo (8 RS)	442	110.50	30	4.43195	3.315	7.74695	70.11	**7.83590**	Yes

Table 1: Study A, serial workload, control_scale = 1. All three designs are correct (functional oracle PASS) and meet the 140 ns (560-cycle) deadline.



Figure 1: Execution time of each architecture for both workloads, compared with the 140 ns and 70 ns deadlines.

(b) Recommendation: In-order. All three designs are correct and feasible, so the decision comes down to period energy. In-order has the lowest (5.695 nJ). That is 0.636 nJ (10.0%) less than Scoreboard and 2.140 nJ (27.3%) less than Tomasulo, and it still has 21.75 ns (87 cycles) of slack before the deadline. The out-of-order designs are only 31 cycles (6.6%) faster because this workload is limited by the F0 recurrence. Each sample's chain FADD → FMUL → FADD → FDIV costs (3+1)+(5+1)+(3+1)+(12+1) = 27 cycles, since each step adds one publication/wake-up cycle. 16 × 27 cycles plus a 10-cycle start-up for the first LD gives exactly the 442 cycles that both Scoreboard and Tomasulo reach. Dynamic scheduling cannot beat this dependence bound. It only hides the LD and IADD work, which In-order already overlaps after the G1 repair; In-order loses about 2 cycles per sample because writeback stays in program order. That small speed gain is paid for with a higher background power during every cycle (24 mW and 30 mW versus 20 mW) and with costlier control events. Because the module sleeps at only 0.10 mW, finishing 7.75 ns earlier recovers less than 1 pJ. The extra speed has no value once the deadline is met, so In-order is preferred.

(c) Energy-component comparison: In-order versus Tomasulo (from `iot_serial_*_s1_energy.csv`).

Component	In-order count × pJ/event	In-order (pJ)	Tomasulo count × pJ/event	Tomasulo (pJ)	Difference (pJ)
scheduler_entry_probes	1,668 × 0.12	200.16	3,594 × 0.30	1,078.20	+878.04
tag comparisons (producer / CDB)	327 bits × 0.06	19.62	4,200 bits × 0.06	252.00	+232.38
entry_transitions	463 × 0.25	115.75	463 × 0.75	347.25	+231.50
register_reads	160 × 1.0	160.00	81 × 1.0	81.00	−79.00
register_writes	96 × 1.2	115.20	35 × 1.2	42.00	−73.20
ops, admission, result bus, deliveries	2,631.50 (identical)		2,631.50 (identical)		0
Dynamic total		**3,242.23**		**4,431.95**	**+1,189.72**

Table 2: Selected energy components for the serial workload at control_scale = 1.

Calculation of one component. Tomasulo scheduler probes: E = 3,594 events × 0.30 pJ/event = **1,078.20 pJ**. For In-order it is 1,668 × 0.12 = 200.16 pJ. Tomasulo probes more entries because 8 RS entries stay busy while instructions wait for the recurrence, and each probe costs 2.5× more. The CDB tag comparator checks every pending source tag and every active RAT entry at the full 4-bit width (4,200 bits = 1,050 whole-tag compares), whether or not they match. Tomasulo does save register-file energy: operands arrive on the CDB

instead of from the RF, and G6 suppresses 61 obsolete writes to the repeatedly renamed scratch registers. That saving (–152.2 pJ) is far smaller than the control overhead. Probes, tags and transitions together explain the whole +1,189.72 pJ dynamic difference (878.04 + 232.38 + 231.50 – 79.00 – 73.20 = 1,189.72 pJ). Adding the background difference (30 × 110.5 – 20 × 118.25 = +950.00 pJ) and the slightly longer sleep (+0.775 pJ) reproduces the measured period-energy gap of 2,140.50 pJ. Scoreboard against In-order shows the same pattern on a smaller scale: probes +215.34 pJ, transitions +115.75 pJ, tags +11.52 pJ, WAR comparisons +5.76 pJ (192 bits × 0.03), giving +348.37 pJ dynamic, plus +287 pJ background.

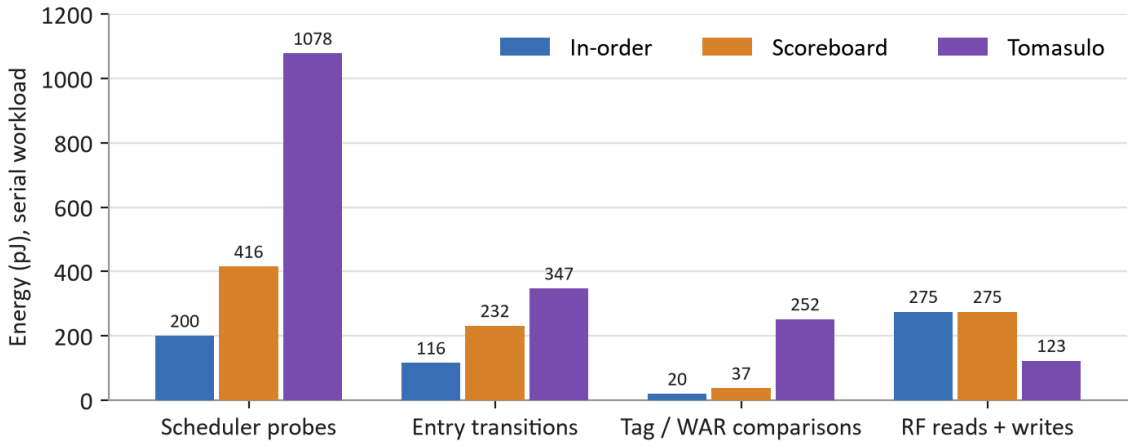


Figure 2: Control-related energy components for the serial workload. Arithmetic, memory, admission and result-bus energy are identical for all three designs and are not shown.

Study B: Four independent streams (iot_four_streams.asm)

(a) Measured results. The 70 ns column uses deadline_70_met, which is the same as comparing time_ns with 70 (280 cycles).

Architecture	Cycles	Time (ns)	E_task (nJ)	E_period (nJ)	Meets 140 ns?	Meets 70 ns?
In-order	473	118.25	5.59328	**5.68145**	Yes	**No**
Scoreboard	328	82.00	5.44390	**5.53570**	Yes	**No**
Tomasulo (8 RS)	239	59.75	5.70392	**5.79794**	Yes	Yes

Table 3: Study B, four-stream workload, control_scale = 1.

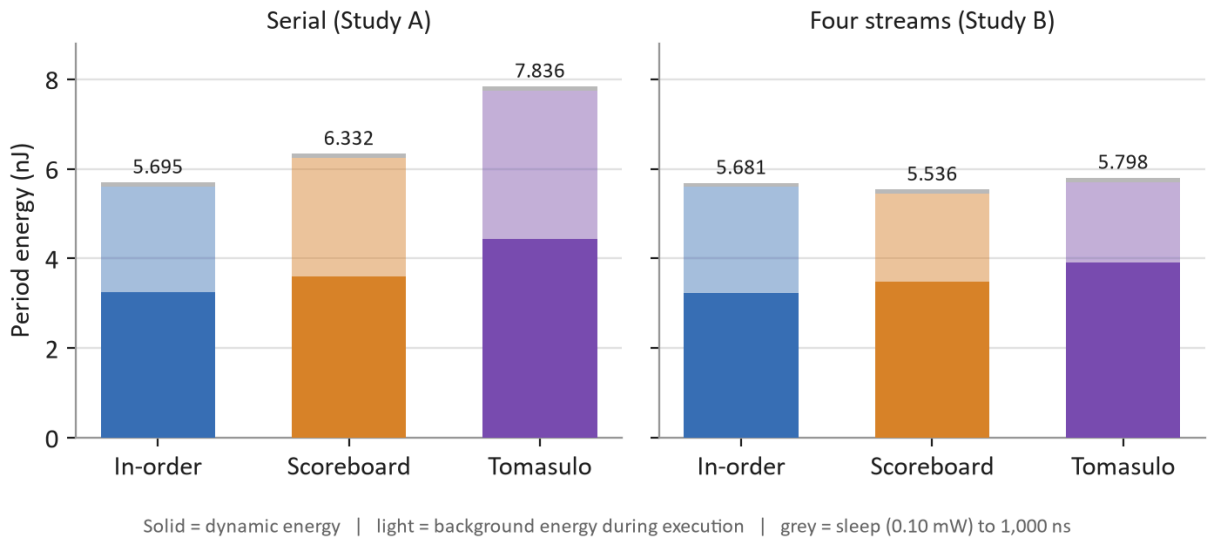


Figure 3: Period energy split into dynamic energy, background energy during execution, and sleep energy for the rest of the

1,000 ns period.

(b) 140 ns deadline: Scoreboard. All three designs are feasible, and Scoreboard has the lowest period energy (5.536 nJ), against 5.681 nJ for In-order and 5.798 nJ for Tomasulo. With four independent recurrences Scoreboard overlaps the streams and finishes in 82 ns instead of 118.25 ns. Its background energy (24 mW × 82 ns = 1,968 pJ) is 397 pJ lower than In-order's (20 mW × 118.25 ns = 2,365 pJ), more than paying for its 248 pJ of extra dynamic control energy. In-order takes exactly the same 473 cycles as on the serial program: in-order writeback makes each completed LD/IADD wait behind the previous 12-cycle FDIV, and only one instruction may wait to launch, so the independence between streams is never exposed. Tomasulo is fastest (59.75 ns) but its 30 mW background and heavier control events (3,911 pJ dynamic) make it the most expensive at this deadline.

(c) 70 ns deadline: Tomasulo. Only Tomasulo is feasible (239 cycles = 59.75 ns, 41 cycles of slack). Scoreboard (82 ns) and In-order (118.25 ns) miss the deadline, so they are excluded before energy is compared. Scoreboard stalls issue on WAW hazards because all streams reuse the scratch names F8–F11, and its rows/FUs are reserved from issue to write. Tomasulo renames these registers through the RAT, admits instructions into free RS entries even when the FU is busy (G4), and lets results reach consumers through the CDB. It approaches the FDIV-unit throughput limit: each FDIV holds the non-pipelined unit for 13 cycles (12 + publication), and 16 × 13 = 208 cycles. Tomasulo is selected at 5.798 nJ per period. That costs 0.262 nJ (4.7%) more than Scoreboard would at 140 ns, which is the price of the tighter deadline.

Study C: When would a different architecture be recommended?

- **A tighter deadline for the serial workload.** In-order stops being feasible below 118.25 ns (473 cycles). Between 110.5 and 118.25 ns, Scoreboard is the choice: it is as fast as Tomasulo and uses 1.504 nJ less per period. Below 110.5 ns no baseline design is feasible, because 442 cycles is the recurrence bound (16 × 27 + 10). Meeting such a deadline would need lower FP latencies, pipelined units or a different algorithm *(hypothesis)*.
- **More independent work (ILP).** Measured: changing from one recurrence to four independent streams moves the choice from In-order to Scoreboard (140 ns) or Tomasulo (70 ns). More independent streams, or workloads whose reused scratch names cause frequent WAW/WAR stalls, would favour Tomasulo's renaming further *(hypothesis)*.
- **Lower controller cost.** Timing does not depend on control_scale, so only control energy changes. The verified serial rows keep In-order cheapest at every scale (table below). For the four-stream workload, supplementary standalone runs (UNVERIFIED, no reference signature) show Tomasulo becoming cheapest at 140 ns when control_scale ≤ 0.6: 5.050 vs 5.095 nJ for Scoreboard at 0.6, but 5.237 vs 5.205 nJ at 0.7. The crossover calculated from the event counts is about 0.66. A cheaper scheduler implementation would therefore change the four-stream recommendation.
- **Higher sleep power (race-to-idle).** A faster design saves P_sleep × ΔT in the idle part of the period. Tomasulo would beat Scoreboard on four streams once P_sleep ≥ 260.02 pJ / 22.25 ns ≈ 11.7 mW. Scoreboard would beat In-order on the serial workload only above 635.37 pJ / 7.75 ns ≈ 82 mW. With the specified 0.10 mW, idle time is almost free and background power while running dominates.
- **RS capacity.** Supplementary UNVERIFIED runs: with 4 RS entries Tomasulo saves 3 mW of background power, but the four-stream run slows to 336 cycles (84 ns, missing 70 ns) and costs 6.082 nJ. With 16 entries it reaches 234 cycles (58.5 ns) but costs 6.678 nJ. On the serial workload even 4 RS entries (6.971 nJ) cannot match In-order. Eight entries is a reasonable balance.
- **Unmodelled effects** *(hypotheses)*: variable-latency loads or cache misses, branches with speculation, or pipelined FUs would increase the value of out-of-order latency hiding and could favour Tomasulo even for less parallel workloads.

control_scale (serial, verified)	In-order E_period (nJ)	Scoreboard E_period (nJ)	Tomasulo E_period (nJ)

0.5	5.38964	5.63835	6.31442
1	5.69541	6.33155	7.83590
2	6.30694	7.71795	10.87885

Table 4: Verified sensitivity rows (serial workload, provided only for context). The ranking does not change.

4. Short-answer questions

Q1. Workload and requirements

(a) F0 carries the filter state between samples. `FDIV F0, F11, F26` of sample k writes F0, and `FADD F9, F0, F8` of sample k+1 reads it. This is a loop-carried (cross-sample) **RAW** / **true data dependence**, which forms a serial recurrence. The reused scratch registers F8–F11 also create name (WAR/WAW) dependences.

(b) One cycle at 4 GHz is 1 / (4×10^9 Hz) = 0.25 ns. Cycle budget = 140 ns / 0.25 ns per cycle = 140 ns × 4 cycles/ns = **560 cycles**.

(c) In the four-stream program, samples are assigned round-robin to four independent states F0–F3. Consecutive samples therefore do not depend on each other, and each recurrence repeats only every fourth sample (24 instructions apart), so up to four chains can overlap. The streams still share the five FUs, the scratch names F8–F11 (WAR/WAW name dependences) and the R1 counter chain. It computes different results, so it is not an equivalent rewrite of the serial program.

Q2. A scheduling decision: G4, Tomasulo (tests/cases/load_admission.asm)

I	Assembly	Issue	Start	Done	Write
1	`LD F0, [0]`	1	2	9	10
2	`IADD R1, R2, R3`	2	3	3	4
3	`LD F1, [1]`	**3**	**11**	18	19

(a) Operand state and resources. I3 (`LD F1, [1]`) has no register source (the word index is part of the instruction) and its destination F1 has no pending producer, so it has nothing to wait for. At cycle 3 only two of the 8 RS entries are busy (I1 and I2), so `rs_free` = true. The single MEMORY unit is occupied by I1 from Start 2 through its Write at cycle 10, so `fu_free` = false and `any_inflight` = true.

(b) Why it is admitted. In Tomasulo, entering a reservation station (issue) is separate from launching on an FU. The repaired G4 returns `rs_free`, so I3 issues in cycle 3 even though the MEMORY unit is busy. It then waits in its RS. I1 publishes in cycle 10, and a freed FU cannot be reused in its publication cycle, so I3 launches in cycle 11 (Done 18, Write 19). The starter G4 also required `fu_free && !any_inflight`, which would have kept I3 (and before it I2) out of the RS until the machine drained.

(c) Evidence. Test `tests/cases/load_admission.asm`, instruction **I3**: Issue = 3, while I1 still holds the MEMORY unit (Start 2, Write 10). I3 Start = 11 = I1 Write + 1. See screenshot S4.

Q3. Producer tags and register versions

(a) Wait. A nonzero producer tag (3) means the register file holds a stale value and producer 3 will supply the version this consumer needs, so the consumer records tag 3 (G5: `producer_tag != 0`).

(b) When producer 3 broadcasts on the CDB, every waiting source whose tag is 3 compares equal, captures the broadcast value, and clears its tag to 0. Both operand positions are checked. Each consumer then becomes ready and may launch from the next cycle.

(c) No. The RAT entry is 7, so a younger instruction (tag 7) has since been renamed onto that register and owns its newest version. If producer 3 wrote the register and cleared the RAT, it would overwrite the newer mapping, and later readers would see an old value or never wait for producer 7 (a WAW error). G6 updates the register only when `current_tag == result_tag` ($7 \neq 3$). The value still goes to the tag-3 consumers,

because they need that older version.

Q4. Time, power and energy

> `T = cycles / f = 400 / 4 GHz =` **100 ns**
> `E_background = P_bg × T = 20 mW × 100 ns = 2,000 pJ`
> `E_task = E_dynamic + E_background = 3,000 pJ + 2,000 pJ =` **5,000 pJ = 5.00 nJ**
> `P_task,avg = E_task / T = 5,000 pJ / 100 ns =` **50 mW**
> `E_period = E_task + P_sleep × (period − T) = 5,000 pJ + 0.10 mW × (1,000 − 100) ns = 5,000 + 90 =` **5,090 pJ = 5.09 nJ**

(b) Summing latencies assumes the operations run one after another. In these machines different FUs run in parallel, so elapsed time is set by the critical dependence path plus structural stalls and the issue/publication/wake-up boundaries. For the serial program the latency sum is 16 × (8+3+5+3+12+1) = 512 cycles, yet In-order takes 473 cycles and Tomasulo 442, because the LD and IADD overlap the FP chain. On four streams Tomasulo needs only 239 cycles. The sum can also underestimate time, because it ignores stalls and the extra cycle each publication and wake-up adds.

Q5. Validation and before/after comparison

Architecture (serial, scale 1)	Before: cycles	Before: ns	Before: E_period (nJ)	Before: 140 ns?	After: cycles	After: ns	After: E_period (nJ)	After: 140 ns?	Change
In-order	609	152.25	6.35817	**No**	473	118.25	5.69541	Yes	-136 cyc / -0.663 nJ
Scoreboard	704	176.00	7.69310	**No**	442	110.50	6.33155	Yes	-262 cyc / -1.362 nJ
Tomasulo (8 RS)	704	176.00	8.93364	**No**	442	110.50	7.83590	Yes	-262 cyc / -1.098 nJ

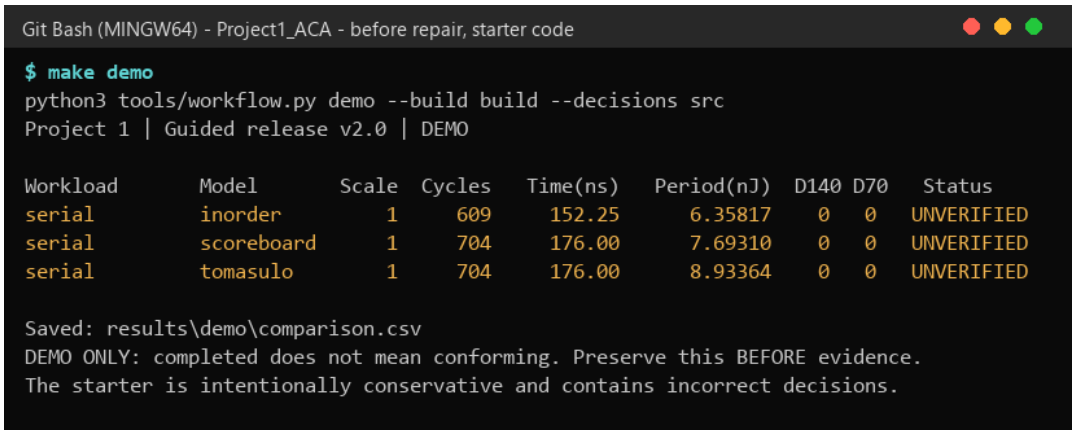
Table 5: Unedited starter (`make demo`, UNVERIFIED, screenshot S1) compared with the final verified result (`make evaluate`, S3).

(a) In-order: before repair, 609 cycles (152.25 ns) and 6.358 nJ per period, missing the 140 ns deadline. After repair, 473 cycles (118.25 ns) and 5.695 nJ, meeting the deadline (–136 cycles, –0.663 nJ, –10.4%).

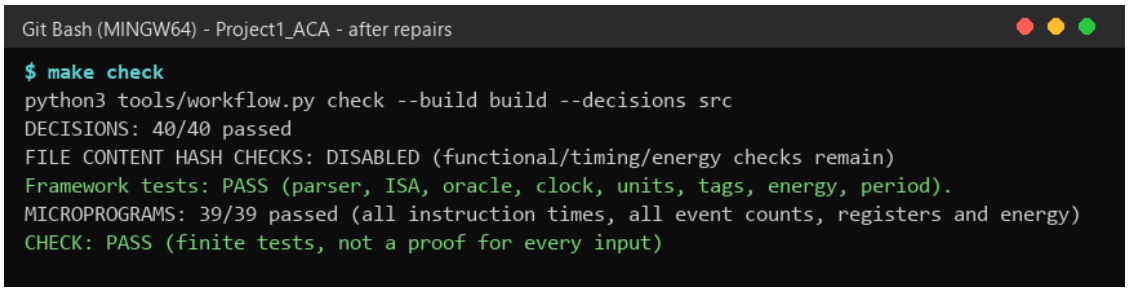
(b) Responsible decision: G1. The starter launched the head instruction only when no older instruction was in flight. In the before-run, IADD (I6) could not start until FDIV (I5) published at cycle 37 (Start 38), and the next LD (I7) started at 40. After the repair, I6 starts at 26 and I7 at 27, overlapping the 12-cycle FDIV on different FUs (screenshots S5a/S5b). The sample-to-sample interval drops from 38 to 29 cycles, and the shorter run cuts background energy (20 mW × 34 ns ≈ 680 pJ). The same over-restriction explains the Scoreboard (G2: `!any_inflight`) and Tomasulo (G4) improvements of 262 cycles each.

5. Run evidence

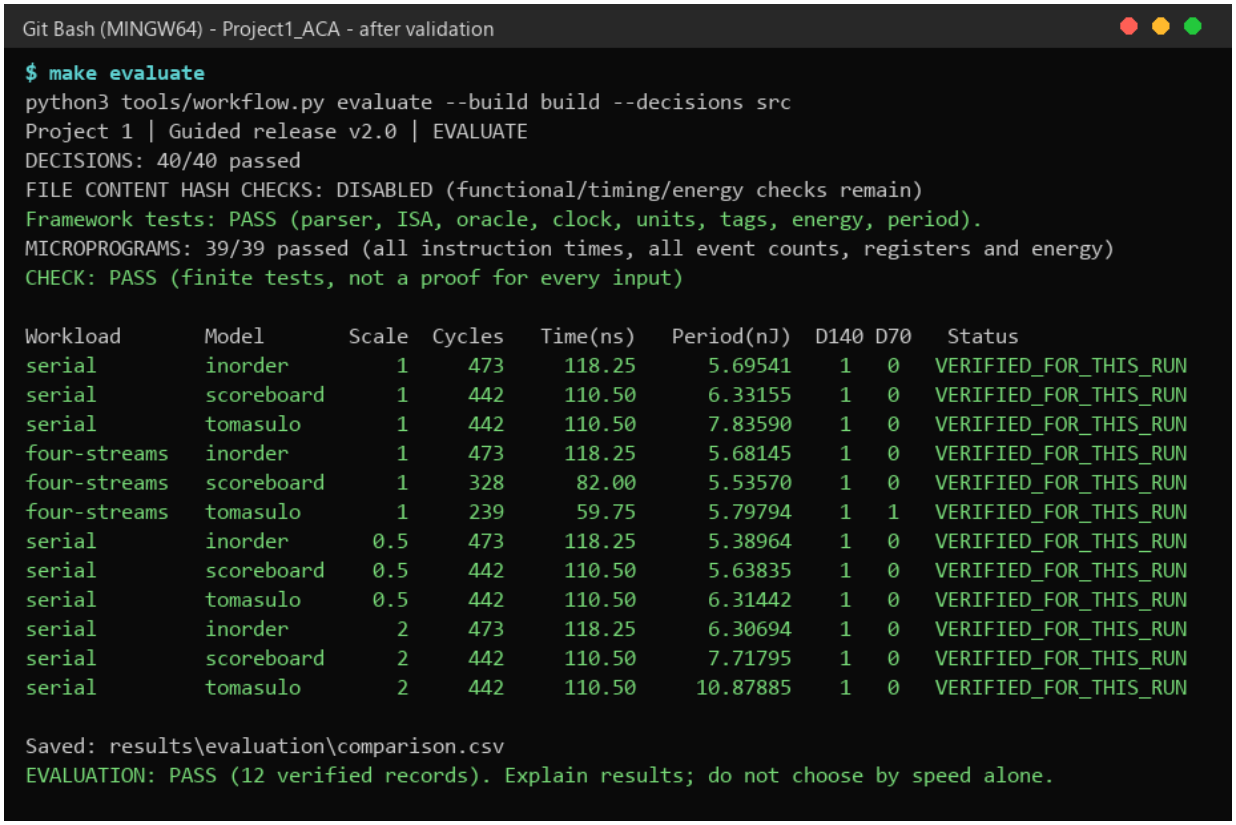
The images below are terminal captures of this project's own runs. Each shows the command and its unmodified output.



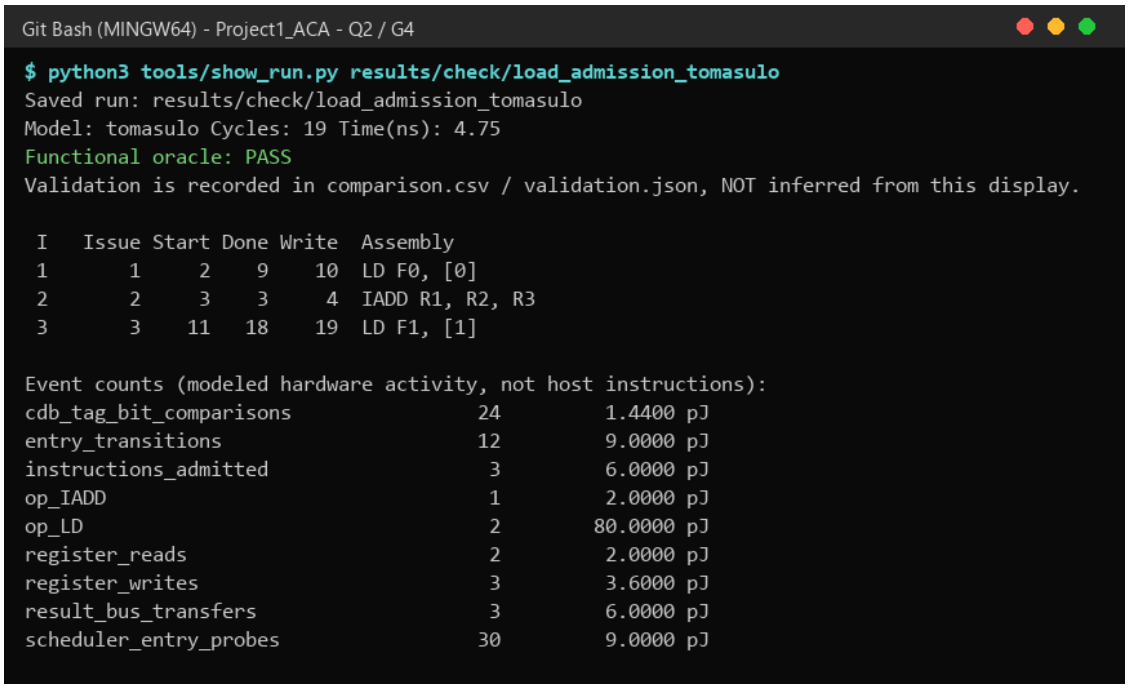
S1: Before repair: `make demo` on the untouched starter. All three models are UNVERIFIED (results preserved in results_before/demo).



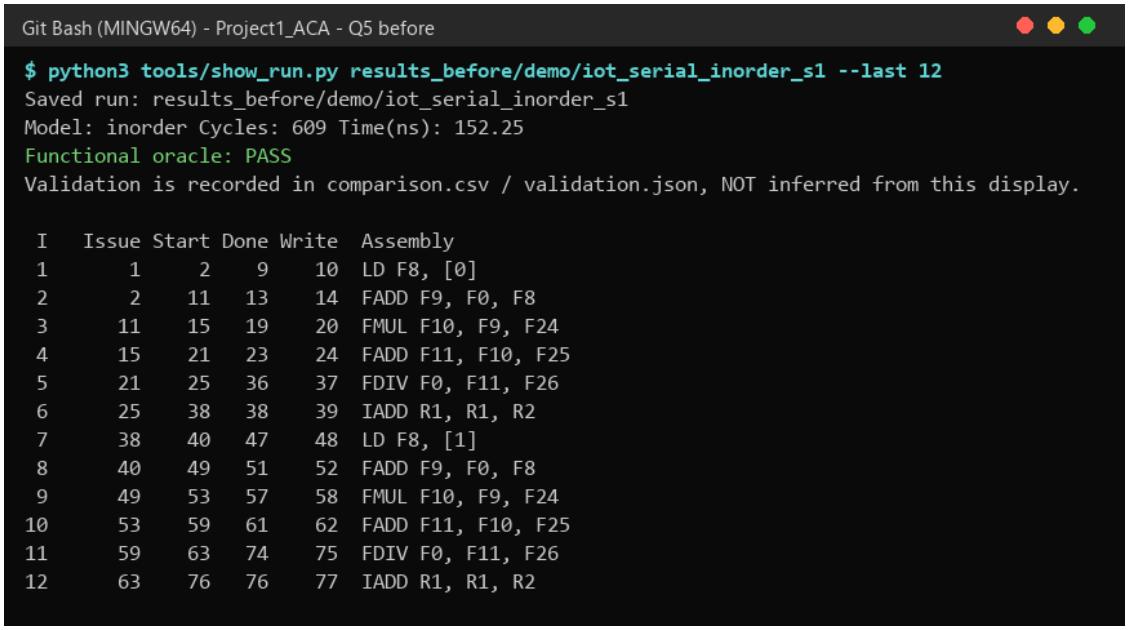
S2: Final validation: `make check`. Decisions 40/40, framework PASS, microprograms 39/39, CHECK: PASS.



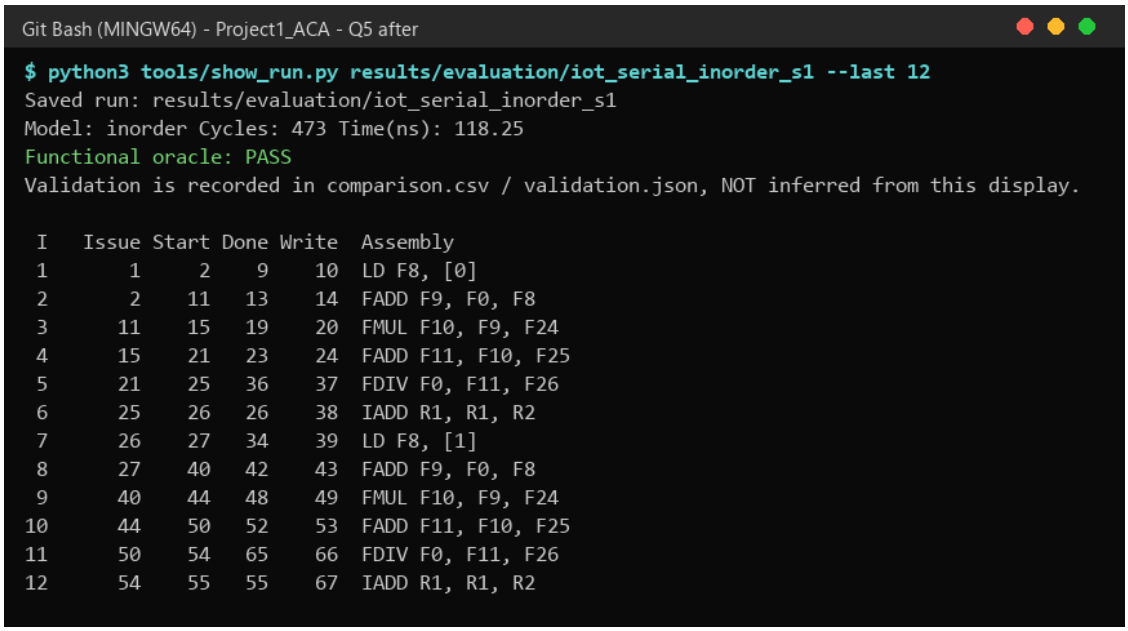
S3: Final evaluation: `make evaluate`. The six control_scale = 1 rows (serial and four-stream) are VERIFIED_FOR_THIS_RUN, with time in ns and period energy in nJ.



S4: Scheduling evidence for Q2 (G4, Tomasulo, load_admission.asm). I3 issues at cycle 3 while I1 holds the MEMORY unit, and starts at 11, after I1 writes at 10.



S5a: Q5 before repair: In-order serial timing (first 12 instructions). I6 starts at 38, after FDIV I5 writes at 37.



S5b: Q5 after repair: I6 starts at 26 and I7 at 27, overlapping FDIV I5 (Start 25, Write 37).

Appendix: Optional student variant

Following `tests/student_cases/README.md`, `my_variant.asm` copies load_admission.asm with `.mem 0 7.5` changed to `.mem 0 10.0`. *Prediction:* only F0 (loaded from word 0) changes, from 7.5 to 10.0. Timing does not change, because latencies and dependences do not depend on data values. *Result* (standalone Tomasulo run, UNVERIFIED because a custom input has no reference signature): F0 = 10, F1 = 2.5, R1 = 7. Timing is identical (I1 1/2/9/10, I2 2/3/3/4, I3 3/11/18/19; 19 cycles), as predicted.